

# Grid dimensioning and data formatting
# Advanced computer architecture

(B-KUL-JLI1QS)

March 7, 2024




| | |
|---|---|
| Session Date: | March 1, 2024 |
| Partners: | Emma Dejagere |
| | Brecht Van de Sijpe |
| Class: | MElsw |
| Instructor: | Alex Marinsek |

# 1 Introduction

This labsession focusses on grid dimensioning and data formatting. For this two kernel's are implemented. The first kernel inverts the colors of an image. The second kernel uses augmentation so that the red component $x$ is transformed like $x = (x \mod 25) \cdot 10, x \in (100, 200)$.

# 2 Method

The invert-kernel calculates the current array index based on the threadId and the blockdimension: int index = blockIdx.x * blockDim.x + threadIdx.x. The value on this arrayindex is substracted from 255 and this becomes the new value in the image-array. Aditional checks to make sure that this index is inbetween the array bounds are made.

For the augmentation-kernel additional lines are required. For this it is first checked if the arrayindex can be divided by three, if so the calculation for the red component is performed. When this is not the case the color is inverted like in the first version.

# 3 Results

In figure 1 and figure 2 the performance of the program can be found. On the horizontal axis the amount of threads in a block can be seen and vertically the calculation time in nanoseconds. For each measurement the kernel is executed 1000 times and the average of these results is stored.

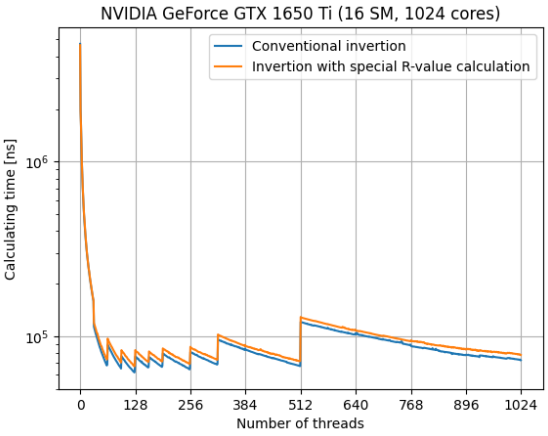



Figure 1: Threads per block results on NVIDIA GeForce GTX 1650 Ti

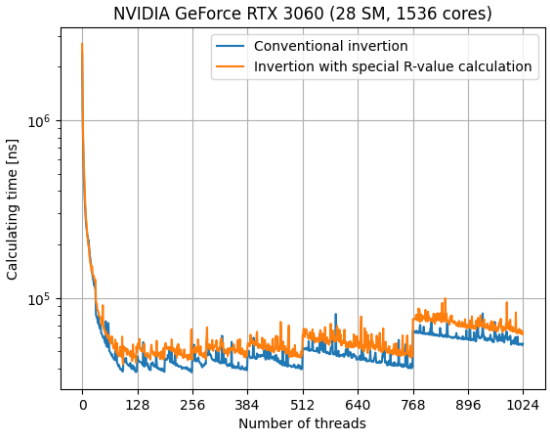



Figure 2: Threads per block results on NVIDIA GeForce RTX 3060

# 4 Conclusion

Using a bigger blocksize overall leads to a better performance result with a lower calculation time. The figures show that partially filled warps perform worse. When the amount of threads in a block is over 32, and a warp is partially filled the GPU performs a lot less efficiënt. Because



the partially filled warp occupies 32 cores, but does not use it's full computing power,this is where the divergence can be seen in the graph. Since we use a specific blocksize, a lot of threads can be partially filled when the blocksize is a little bit over the size of a warp. Because a big part of all these warps is not fully utilized the graph makes a jump up (higher calculation time) and gradually lowers since all those warps are being utilized more when the blocksize grows.